

Estilização de Componentes no React

Sumário:

1. CSS tradicional e Estilos inline.....	2
1.1 CSS Tradicional (Folhas de Estilo).....	2
1.2 Estilos Inline (Inline Styles).....	9
2. CSS Modules e Styled Components.....	15
2.1 CSS Modules.....	15
2.2 Styled Components.....	18
3. Estilização Condicional no React.....	21
3.1 Estilização Condicional com Estilos Inline (Inline Styles).....	21
3.2 Estilização Condicional com Classes CSS.....	23
3.3 Estilização Condicional com CSS Modules.....	25
3.4 Estilização Condicional com Styled Components.....	27
4. Criação de Temas e Personalização de Interfaces em React.....	30
4.1 Utilizando Styled Components para Criar Temas.....	30
4.2 Alternando Temas com Context API.....	33
Referências.....	36

1. CSS tradicional e Estilos inline

No React, a estilização de componentes pode ser feita de várias maneiras, dependendo das preferências e requisitos do projeto. A seguir, temos algumas das principais abordagens de estilização em componentes React.

1.1 CSS Tradicional (Folhas de Estilo)

No React, o CSS tradicional funciona de maneira semelhante ao que é usado em aplicações web padrão, onde os estilos são definidos em arquivos com **extensão .css** e aplicados via classes ou IDs HTML. A única diferença importante é que, no React, os arquivos de estilo são importados diretamente nos componentes, uma vez que o JSX, que mistura HTML e JavaScript, exige esse tipo de integração.

1.1.1 Exemplo Básico de Uso

Abaixo está um exemplo básico de como usar arquivos CSS tradicionais em um componente React. No quadro abaixo, segue um código CSS que adicionamos no arquivo **App.css**, por exemplo:

```
.container {  
  background-color: lightblue;  
  padding: 20px;  
  border-radius: 10px;  
}  
.text {  
  color: white;  
  font-size: 20px;  
}
```

O código CSS acima define estilos para duas classes. A classe **.container** aplica um fundo azul claro, espaçamento interno de 20 pixels e bordas arredondadas de 10 pixels aos elementos que a utilizam. Já a classe **.text** define a cor do texto como branca e o tamanho da fonte como 20 pixels, estilizando os elementos que possuem essa classe.

Em seguida, em um componente React, vamos importar o arquivo CSS e aplicar as classes aos elementos HTML usando o atributo **className** (note que o JSX usa **className** em vez de **class** para evitar conflito com a palavra-chave **class** do JavaScript).

Desta forma, considere alterar o arquivo **App.jsx** com o seguinte código a seguir:

```
// App.jsx
import "../App.css"; // Importa o arquivo CSS
function App() {
  return (
    <div className="container">
      <p className="text">Este é um texto estilizado com CSS tradicional.</p>
    </div>
  );
}
export default App;
```

Neste exemplo:

- O arquivo **App.css** define duas classes, **container** e **text**, que são aplicadas no componente React.
- O **className="container"** e **className="text"** no JSX referenciam as classes CSS, e o arquivo CSS está sendo importado no topo do arquivo JavaScript.
- O React usa essas classes para aplicar os estilos da mesma forma que em uma página HTML tradicional.

Veja na imagem abaixo a execução desta aplicação:

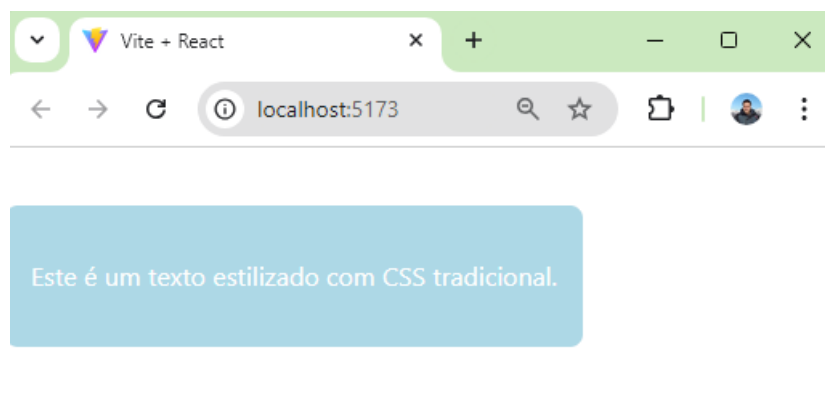


Figura 1: Exemplo de uso de CSS Tradicional com React

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No corpo da página, há uma caixa retangular azul clara com o texto "Este é um texto estilizado com CSS tradicional" em branco.

1.1.2 Organização dos Arquivos CSS

Para projetos React mais estruturados, é importante manter os arquivos CSS organizados de maneira eficiente. Neste contexto, algumas práticas comuns de uso de CSS incluem: uso de CSS Global, uso de CSS específico para componentes ou uma definição melhor da estrutura de pastas CSS.

1.1.2.1 CSS Global

Um arquivo CSS global que contém estilos que se aplicam a toda a aplicação, como fontes padrão, cores e layouts gerais. Por exemplo, considere o seguinte código para o arquivo **index.css**:

```
/* index.css */
body {
  margin: 0;
  padding: 0;
  font-family: 'Arial', sans-serif;
}
h1, h2, h3 {
  color: rgb(32, 105, 209);
}
```

Esse código CSS define estilos básicos para uma página web. O elemento **body** tem margens e padding zerados, e a fonte utilizada será **'Arial'** ou, em sua ausência, uma fonte genérica sem serifa (**sans-serif**). Para os elementos **h1**, **h2** e **h3**, a cor do texto será definida como um tom de azul claro (**rgb(32, 105, 209)**).

Agora, no arquivo **main.jsx**, você importa o arquivo **index.css** para que os estilos sejam aplicados globalmente. Além disso, renderiza os componentes da aplicação no DOM, tal como apresentado no quadro abaixo:

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import OutroApp from './OutroApp.jsx'
import './index.css'
```

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
    <OutroApp />  
  </StrictMode>,  
)
```

Se você ainda não tiver o arquivo **App.jsx**, crie-o para ver os estilos globais aplicados. Este componente será renderizado no DOM. Veja seu código abaixo:

```
// App.jsx  
  
function App() {  
  return (  
    <div>  
      <h1>Bem-vindo ao Meu App</h1>  
      <h2>Este é um exemplo de CSS global no React</h2>  
    </div>  
  );  
}  
export default App;
```

Em seguida, se você ainda não tiver o arquivo **OutroApp.jsx**, crie-o para ver os estilos globais aplicados. Este componente será renderizado no DOM. Veja seu código abaixo:

```
// OutroApp.jsx  
  
function OutroApp() {  
  return (  
    <div>  
      <h3>Este é outro exemplo de CSS global no React</h3>  
    </div>  
  );  
}  
export default OutroApp;
```

Veja na imagem abaixo a execução desta aplicação:



Figura 2: Exemplo de uso de CSS Global com React

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". o corpo da página, há o texto "Bem-vindo ao Meu App" em azul, seguido pelas frases "Este é um exemplo de CSS global no React" e "Este é outro exemplo de CSS global no React", ambas em azul. O fundo da página é branco.

1.1.2.2 CSS Específico para Componentes

Para manter o código modular, pode-se criar um arquivo CSS separado para cada componente. Isso evita a poluição no escopo global e facilita a manutenção do código. Por exemplo, um arquivo chamado **component.css** com o seguinte estilo CSS:

```
.componente {  
  background-color: lightblue;  
  padding: 15px;  
}
```

Em seguida, podemos utilizar este estilo CSS em um componente React definido em **Component.jsx** da seguinte forma:

```
import './component.css';  
function Component() {  
  return <div className="componente">Estilo para componente</div>;  
}  
export default Component;
```

1.1.2.3 Estrutura de Pastas CSS

Em projetos maiores, é comum organizar os arquivos CSS dentro de uma estrutura de pastas, separando estilos globais, estilos de componentes e até estilos de páginas específicas. Observe na ilustração abaixo um exemplo de organização dos diretórios e arquivos de um projeto em React:

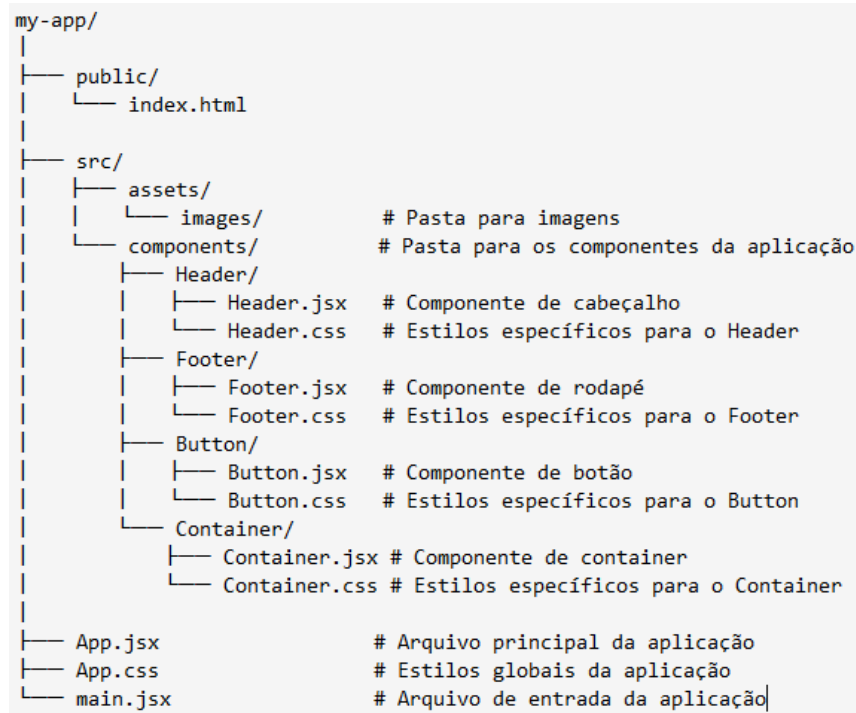


Figura 3: Exemplo estrutura de pastas de um projeto em React

Descrição: A imagem mostra a estrutura de diretórios de um projeto chamado "my-app" em formato de árvore. Dentro da pasta "public/", há o arquivo "index.html". Na pasta "src/", há uma subpasta "assets/" que contém uma pasta "images/", e uma subpasta "components/" com as pastas "Header/", "Footer/", "Button/" e "Container/", cada uma contendo arquivos JSX e CSS específicos, como "Header.jsx" e "Header.css". Na raiz de "src/", há os arquivos "App.jsx", "App.css" e "main.jsx", representando o código principal e os estilos globais da aplicação.

1.1.3 Escopo Global vs. Escopo Local

Uma característica importante dos arquivos CSS tradicionais é que eles têm **escopo global**. Isso pode gerar conflitos de estilo quando diferentes componentes compartilham o mesmo nome de classe, mas precisam de estilos diferentes.

Veja abaixo um exemplo deste tipo de conflito, onde definimos a classe CSS chamada **.button** em um arquivo chamado **App.css**:

```
/* App.css */  
.button {  
  background-color: blue;  
  color: white;  
}
```

E, depois, definirmos outra classe chamada **.button** em um outro arquivo chamado **AnotherComponent.css**:

```
/* AnotherComponent.css */  
.button {  
  background-color: red;  
  color: black;  
}
```

Se ambos os componentes (**App** e **AnotherComponent**) forem renderizados na mesma página, os estilos podem se sobrepor, causando resultados inesperados.

Para evitar esses problemas, você pode seguir algumas práticas, tais como Naming Convention (BEM) ou CSS Modules.

1.1.3.1 Naming Convention (BEM)

Use convenções de nomenclatura como BEM (Block, Element, Modifier) para garantir que os nomes de classe sejam exclusivos e descritivos. Isso ajuda a evitar conflitos e torna os estilos mais fáceis de manter. Veja no quadro abaixo um exemplo de Naming Convention (BEM):

```
/* Header.css */  
.header__title {  
  font-size: 24px;  
  color: blue;  
}  
.header__button--primary {  
  background-color: green;  
  color: white;  
}
```


1.1.3.2 CSS Modules

Uma solução mais avançada seria usar CSS Modules, que permite que os estilos sejam escopados localmente aos componentes, resolvendo completamente o problema de conflitos de classe. Embora os CSS Modules não sejam exatamente CSS tradicional, eles permitem que você escreva CSS como de costume, mas com a vantagem do escopo local.

O nome do arquivo de um CSS Module deve seguir o padrão **<nome>.module.css**. Por exemplo, considere o arquivo **Button.module.css** com o seguinte código:

```
.button {  
  background-color: blue;  
  color: white;  
  padding: 10px 20px;  
  border-radius: 5px;  
}
```

Podemos utilizar este estilo, por exemplo, no arquivo **Button.jsx** da seguinte forma:

```
import styles from "./Button.module.css"; // Importa os estilos  
  
const Button = () => {  
  return <button className={styles.button}>Click Me</button>; // Usa o estilo  
};  
  
export default Button;
```

Neste exemplo, **styles.button** acessa a classe **button** definida em **Button.module.css**. O React vai gerar um nome de classe único (como **Button_button__2xQNz**), tornando impossível que outra classe **.button** de outro arquivo sobrescreva esse estilo. Veremos mais sobre CSS Modules no decorrer deste curso.

1.2 Estilos Inline (Inline Styles)

O uso de Estilos Inline (Inline Styles) no React é uma forma de **aplicar estilos diretamente aos elementos HTML** utilizando objetos JavaScript. Isso significa que, em vez de escrever um arquivo CSS separado ou usar classes, você define os estilos como um objeto dentro do componente e passa esse objeto diretamente para o atributo **style** do elemento.

Essa abordagem é útil quando você quer aplicar estilos de maneira rápida ou condicional diretamente no código do componente, sem a necessidade de criar arquivos CSS externos.

1.2.1 Estrutura de Estilos Inline

No React, o valor do atributo **style** deve ser um objeto JavaScript, no qual as propriedades CSS são escritas em camelCase (ao contrário do uso de hífen no CSS tradicional). Além disso, os valores numéricos não requerem unidades como **px**, pois o React assume **px** como padrão, exceto quando uma unidade diferente é especificada, como % ou **em**.

OBS: Saiba que **em** é uma unidade de medida relativa usada em CSS, comumente empregada para definir o tamanho de fontes e outros elementos. Ela é relativa ao tamanho da fonte do elemento pai. Por exemplo, se o tamanho da fonte do elemento pai for **16 pixels**, então **1em** equivale a **16 pixels**. Se você definir um valor como **2em**, isso será o dobro, ou seja, **32 pixels**. A unidade em é útil para criar layouts e textos responsivos, pois permite que o tamanho dos elementos se ajuste de acordo com o contexto, mantendo uma proporção relativa. Além de fontes, **em** também pode ser usada para margens, espaçamentos, entre outros estilos de layout.

1.2.2 CamelCase nas Propriedades CSS

No React, como o código JSX é basicamente JavaScript, as propriedades de estilo que normalmente usam hífen no CSS (como **background-color** ou **font-size**) precisam ser transformadas para a notação **camelCase**.

Veja abaixo alguns exemplos de conversão:

Propriedade CSS Tradicional	Propriedade no React (CamelCase)
background-color	backgroundColor
font-size	fontSize
margin-top	marginTop
border-radius	borderRadius

1.2.3 Exemplo Básico

Veja abaixo um exemplo simples de como aplicar estilos inline no React:

```
function App() {  
  const estiloDiv = {  
    backgroundColor: "lightblue",  
    padding: "20px",  
    fontSize: 18,  
  };  
  return <div style={estiloDiv}>Meu Componente Estilizado</div>;  
}  
export default App;
```

Neste exemplo:

- O objeto **estiloDiv** contém as propriedades de estilo. Cada propriedade CSS que normalmente usaria no CSS tradicional (como **background-color**) é convertida para a forma camelCase (**backgroundColor**).
- O valor **20px** é passado como uma string porque inclui a unidade **px**.
- O valor de **fontSize** é apenas um número, de modo que o React assume **px** como unidade padrão.
- O atributo **style** no JSX recebe o objeto **estiloDiv** que contém os estilos que serão aplicados diretamente ao elemento **div**.

Veja na figura abaixo o resultado no navegador desta aplicação:

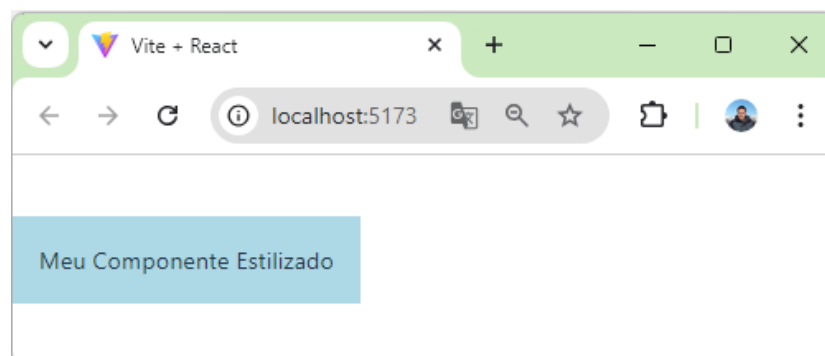


Figura 4: Exemplo simples de como aplicar estilos inline no React

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No corpo da página, há uma caixa retangular azul com o texto "Meu Componente Estilizado".

1.2.4 Valores Numéricos e Unidades

No React, você pode omitir unidades como **px** ao definir valores numéricos. Se o valor for um número puro, o React assume que é **px**, exceto para algumas propriedades específicas (como **lineHeight**, onde o número pode ser unitário).

Veja um exemplo no quadro abaixo:

```
function App() {  
  const estiloDiv = {  
    backgroundColor: "lightblue",  
    padding: 20, // React assume que isso é 20px  
    marginTop: 30, // Isso também será 30px  
  };  
  return <div style={estiloDiv}>Meu Componente Estilizado</div>;  
}  
export default App;
```

Neste mesmo exemplo, se quiser usar uma unidade diferente (por exemplo, % ou **em**), você precisa colocar o valor como uma string:

```
function App() {  
  const estiloDiv = {  
    backgroundColor: "lightblue",  
    width: "50%", // Unidades como % devem ser strings  
    fontSize: "1.5em", // Unidades em 'em' também precisam ser strings  
  };  
  return <div style={estiloDiv}>Meu Componente Estilizado</div>;  
}  
export default App;
```

1.2.5 Estilos Inline Múltiplos e Combinação de Objetos

Outra vantagem dos estilos inline no React é a possibilidade de combinar objetos de estilo facilmente, seja com o uso do **operador spread (...)** ou usando funções utilitárias de JavaScript como **Object.assign**.

Você pode usar o operador **spread (...)** para combinar dois ou mais objetos de estilo, conforme exemplo abaixo:

```
function App() {  
  const estiloBase = {  
    padding: "20px",  
    color: "white",  
  };  
  const estiloExtra = {  
    backgroundColor: "blue",  
    fontWeight: "bold",  
  };  
  return (  
    <div style={{ ...estiloBase, ...estiloExtra }}>Estilizado com Spread</div>  
  );  
}  
export default App;
```

Veja na imagem abaixo a execução desta aplicação:

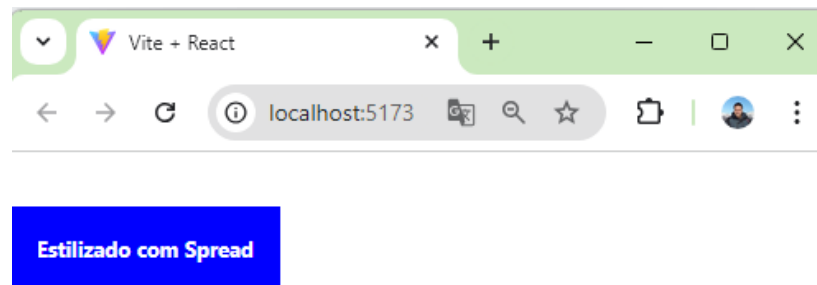


Figura 5: Exemplo de estilos inline com o operador spread (...)

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No corpo da página, há um botão retangular azul escuro com o texto branco "Estilizado com Spread".

Outra maneira de combinar estilos é usando **Object.assign**, conforme quadro abaixo:

```
function App() {  
  const estiloBase = {  
    padding: "20px",  
    color: "white",  
  };  
  const estiloExtra = {  
    backgroundColor: "blue",  
    fontWeight: "bold",  
  };  
  return (  
    <div style={{ ...estiloBase, ...estiloExtra }}>Estilizado com Spread</div>  
  );  
}  
export default App;
```

```
const estiloCombinado = Object.assign({}, estiloBase, estiloExtra);
return <div style={estiloCombinado}>Estilizado com Object.assign</div>;
}
export default App;
```

1.2.6 Limitações dos Estilos Inline

Apesar de ser uma forma rápida e prática para aplicar estilos diretamente, os Inline Styles no React têm algumas limitações:

- **Falta de Pseudo-Classes:** Não é possível aplicar pseudo-classes como **:hover**, **:focus** ou **:active** diretamente nos estilos inline.
- **Falta de Suporte para Pseudo-Elementos:** Não é possível utilizar pseudo-elementos como **::before** e **::after** diretamente.
- **Manutenção e Reusabilidade:** À medida que a complexidade do projeto cresce, o uso excessivo de estilos inline pode se tornar difícil de manter.
- **Complexidade de Media Queries:** Aplicar media queries diretamente nos estilos inline é inviável. As Media queries permitem aplicar estilos diferentes a uma página com base nas características do dispositivo ou da janela do navegador, como largura, altura, resolução, orientação (paisagem ou retrato) e outros.

1.2.7 Quando Usar Estilos Inline?

Os Estilos Inline (Inline Styles) são particularmente úteis em cenários onde:

- Você precisa aplicar um estilo específico e dinâmico baseado em uma condição.
- O componente é muito pequeno e não precisa de reutilização extensiva dos estilos.
- Deseja evitar criar arquivos CSS adicionais ou uma estrutura complexa de classes.

2. CSS Modules e Styled Components

2.1 CSS Modules

CSS Modules são uma forma de escrever estilos CSS em que os nomes de classes e identificadores são escopados localmente ao componente onde são usados, evitando conflitos de nomes globais. Essa abordagem é muito útil em projetos React, pois permite aplicar estilos de maneira modular, tornando o código mais fácil de manter e escalável, especialmente em aplicações grandes com muitos componentes.

Com CSS Modules, os arquivos CSS são tratados como módulos, e cada classe ou identificador é mapeado para um nome único, geralmente composto pelo nome da classe e um hash gerado automaticamente. Isso garante que os estilos sejam aplicados apenas ao componente certo, sem poluir o escopo global de estilos.

2.1.1 Como Funcionam CSS Modules?

CSS Modules funcionam da seguinte maneira:

- **Escopo Local:** Os estilos definidos no arquivo CSS são automaticamente convertidos em classes únicas, que são aplicadas somente ao componente em que são importados. Isso resolve o problema de colisão de nomes, comum em projetos grandes.
- **Nomes de Classe Únicos:** As classes CSS são renomeadas automaticamente para incluir um hash exclusivo, o que garante que diferentes componentes possam ter classes com o mesmo nome sem interferir uns nos outros.
- **Importação como Objeto:** No React, quando você usa CSS Modules, importa o arquivo CSS como um objeto JavaScript, onde cada classe CSS se torna uma chave do objeto. Isso permite que você aplique classes de forma programática.

2.1.2 Como Usar CSS Modules no React?

Para usar CSS Modules no React, o arquivo CSS deve ser nomeado com a **extensão .module.css**. Isso informa ao React que os estilos nesse arquivo devem ser tratados como módulos, ou seja, com escopo local.

Para exemplificar melhor o uso de CSS Modules em React, considere a criação de um arquivo chamado **App.module.css** com os seguintes estilos:

```
.container {  
  background-color: lightblue;  
  padding: 20px;  
  border-radius: 10px;  
}  
.title {  
  color: darkblue;  
  font-size: 24px;  
  text-align: center;  
}
```

Em seguida, no componente **App.jsx**, você importa o arquivo CSS como um objeto e aplica as classes usando **className**.

```
import styles from "../App.module.css"; // Importa o CSS como um objeto  
function App() {  
  return (  
    <div className={styles.container}>  
      <h1 className={styles.title}>Olá, este é um exemplo de CSS Modules</h1>  
    </div>  
  );  
}  
export default App;
```

Observe neste exemplo:

- **Importação do CSS Module:** No React, você importa o arquivo CSS (**App.module.css**) como um objeto chamado **styles**. Cada classe definida no arquivo CSS se torna uma chave no objeto **styles**.
- **Aplicação de Classes:** Para aplicar a classe **container** e **title** no JSX, você usa **className={styles.container}** e **className={styles.title}**. O React garante que esses estilos sejam aplicados localmente ao componente.
- **Escopo Local:** Mesmo que outro componente no projeto tenha uma classe chamada **container**, os estilos não entrarão em conflito, pois as classes geradas pelo CSS Modules serão únicas.

2.1.3 Como Funciona a Geração de Nomes?

Quando você define uma classe no arquivo CSS, como `container`, o React e o "module bundler" ou "empacotador de módulos" (tal como o Rollup ou o Webpack) transformam essa classe em uma versão única, adicionando um hash ao nome da classe. Por exemplo, a classe `.container` no arquivo `App.module.css` pode ser transformada em algo como:

```
.container_App_module__23KjF {  
  background-color: lightblue;  
  padding: 20px;  
  border-radius: 10px;  
}
```

Essa transformação garante que os estilos aplicados sejam exclusivos ao componente, evitando qualquer possível conflito de estilos.

2.1.4 Benefícios dos CSS Modules

Veja abaixo alguns dos benefícios do uso de CSS Modules:

- **Evita Conflitos de Nomes:** Ao escopar localmente os estilos, CSS Modules evita conflitos de classes que são comuns em grandes projetos com muitos componentes.
- **Manutenção Facilmente Escalável:** Como cada componente tem seus próprios estilos, é mais fácil localizar e alterar o estilo de um componente específico sem quebrar outros.
- **Modularidade e Reutilização:** Você pode criar módulos de estilo que são facilmente reutilizáveis e combináveis em diferentes componentes.
- **Integração Simples:** CSS Modules mantém a sintaxe CSS padrão, o que facilita a adoção por times que já conhecem bem CSS.

2.1.5 Limitações dos CSS Modules

Veja abaixo alguns das limitações do uso de CSS Modules:

- **Overhead de Build:** A construção e a geração dos hash para as classes é um processo adicional que precisa ser configurado e gerenciado pelo Rollup, Webpack ou outra ferramenta de build.

- **Estilos Globais:** Em alguns casos, você ainda pode precisar de estilos globais (como resets de CSS ou temas), o que pode resultar em uma combinação de CSS Modules e folhas de estilo globais, exigindo um bom gerenciamento.
- **Integração com Bibliotecas Externas:** Quando você utiliza bibliotecas de terceiros que já possuem seus próprios estilos globais, como Bootstrap ou Material UI, pode ser um desafio garantir a consistência do escopo.

2.2 Styled Components

[Styled Components](#) é uma biblioteca para React e React Native que permite escrever estilos CSS diretamente dentro dos componentes JavaScript utilizando **Tagged Template Literals**. Essa abordagem é conhecida como **CSS-in-JS** e traz diversas vantagens, como o escopo automático de estilos, a capacidade de aplicar estilos dinâmicos baseados em props e estados, e a remoção de preocupações com conflitos de nomes de classes.

Styled Components permite que você **escreva código JavaScript e CSS juntos**, encapsulando a lógica de estilo diretamente no componente React. Isso resulta em componentes totalmente estilizados e independentes, onde o estilo e a estrutura estão fortemente acoplados, facilitando a modularidade e a reutilização.

2.2.1 Principais Benefícios dos Styled Components

Podemos citar os seguintes benefícios do uso de Styled Components:

- **Escopo Local Automático:** Os estilos são automaticamente aplicados apenas ao componente ao qual pertencem, evitando conflitos de nomes de classes.
- **Estilos Dinâmicos:** Você pode aplicar estilos com base em **props**, estado ou qualquer lógica de JavaScript diretamente dentro do componente.
- **Eliminação de Folhas de Estilo Externas:** Com Styled Components, você não precisa de arquivos CSS separados; os estilos são diretamente integrados no componente.
- **Suporte para Animações e Temas:** A biblioteca também oferece suporte completo para animações e permite a criação de temas para fácil manutenção de estilos globais.

2.2.2 Como Instalar Styled Components

Antes de começar a usar Styled Components em seus projetos, você precisa instalar a biblioteca no seu projeto React:

```
npm install styled-components
```

Ou, se estiver usando yarn:

```
yarn add styled-components
```

2.2.3 Exemplo Básico de Styled Components

Vamos ver um exemplo simples de como usar Styled Components para estilizar um componente React. Desta forma, considere alterar o arquivo **App.jsx** com o seguinte código:

```
import styled from "styled-components";

// Definindo um componente estilizado
const Container = styled.div`
  background-color: lightblue;
  padding: 20px;
  border-radius: 10px;
  text-align: center;
`;

// Definindo outro componente estilizado
const Titulo = styled.h1`
  color: darkblue;
  font-size: 24px;
`;

function App() {
  return (
    <Container>
      <Titulo>Olá, este é um exemplo de Styled Components!</Titulo>
    </Container>
  );
}

export default App;
```

Neste exemplo:

- **styled.div:** Estamos criando um componente estilizado chamado **Container** com a função **styled.div**. Ele retorna uma **div** HTML com os estilos definidos diretamente dentro de um **template literal**.
- **Uso de styled:** **styled** é a função principal da biblioteca. Ela pode ser usada para criar componentes estilizados de qualquer tipo de elemento HTML como **div**, **button**, **input**, etc.
- **Template Literals:** Os estilos CSS são escritos dentro de um template literal (usando crases ``), permitindo interpolação de variáveis JavaScript para criar estilos dinâmicos.

Observe na imagem abaixo a execução desta aplicação:

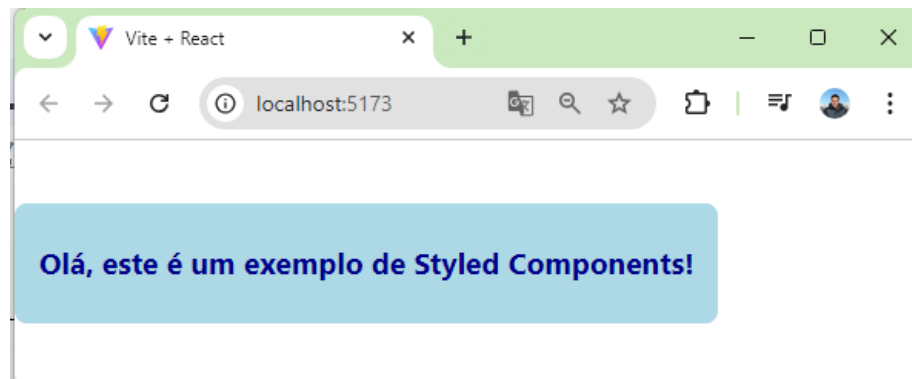


Figura 6: Exemplo de uso de Styled Components

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No corpo da página, há uma caixa retangular azul claro com o texto "Olá, este é um exemplo de Styled Components!" em azul escuro. O fundo da página é branco.

3. Estilização Condicional no React

A estilização condicional permite alterar dinamicamente os estilos de um componente com base em variáveis, estados ou **props** (propriedades). No React, essa funcionalidade é extremamente útil, pois a aparência de um componente pode mudar com base nas interações do usuário, em mudanças no estado, ou em valores que o componente recebe como props.

Existem várias maneiras de implementar estilização condicional no React, dependendo da abordagem que você está usando para aplicar estilos. A seguir veremos cada uma dessas abordagens com exemplos práticos.

3.1 Estilização Condicional com Estilos Inline (Inline Styles)

Com Estilos Inline (Inline Styles) no React, você pode aplicar estilos dinamicamente com base no estado do componente ou em propriedades (**props**). Isso é muito útil para alterar a aparência de um componente em resposta a interações do usuário, como cliques, ou para refletir mudanças de estado.

Veja um exemplo de como alterar a cor de fundo dinamicamente com base no estado:

```
import { useState } from "react";
function App() {
  const [ativo, setAtivo] = useState(false);
  const estiloDiv = {
    backgroundColor: ativo ? "green" : "red", // cor muda dinamicamente
    padding: "20px",
    color: "white",
    textAlign: "center",
  };
  return (
    <div style={estiloDiv} onClick={() => setAtivo(!ativo)}>
      Clique para mudar a cor!
    </div>
  );
}
export default App;
```

Neste exemplo:

- O estado **ativo** é inicializado como **false**.
- O estilo da **div** tem a propriedade **backgroundColor** definida com base no valor de **ativo**. Se ativo for **true**, a cor de fundo será **green**, caso contrário será **red**.
- Ao clicar na **div**, o estado **ativo** muda, e com ele os estilos também mudam.

Veja na imagem abaixo a execução desta aplicação:

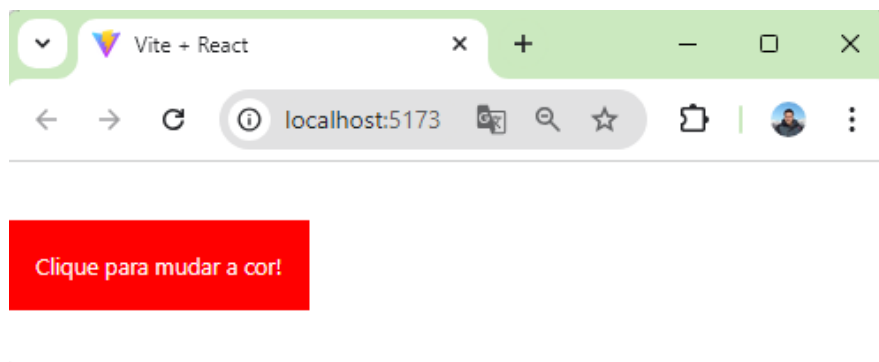


Figura 7: Exemplo sobre a aplicação de estilos inline de forma condicional

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular vermelho com o texto "Clique para mudar a cor!".

Em seguida, clique no botão "Clique para mudar a cor!" e veja o resultado, conforme imagem abaixo:

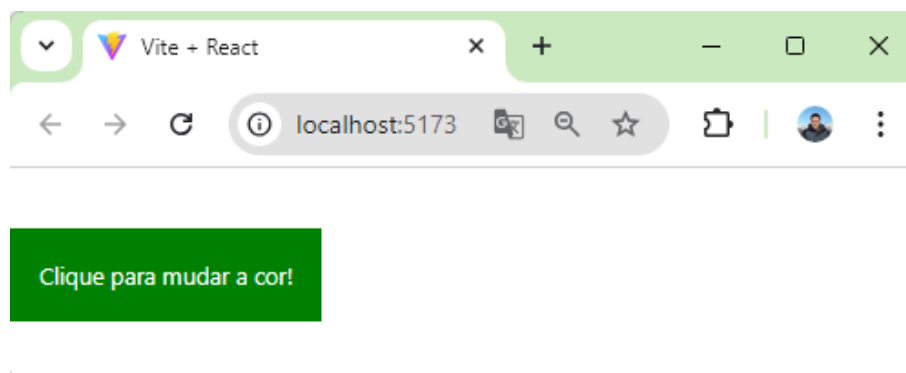


Figura 8: Exemplo após aplicar o estilo de forma condicional

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular verde com o texto "Clique para mudar a cor!".

3.2 Estilização Condicional com Classes CSS

No React, você pode alternar entre diferentes classes CSS com base em condições, como estados ou **props**. Isso permite aplicar estilos de maneira condicional com classes diferentes.

Veremos isso em um exemplo. Primeiro, considere um arquivo CSS chamado **App.css** que contenha com as seguintes classes:

```
/* App.css */
.botao {
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

.ativo {
  background-color: green;
  color: white;
}

.inativo {
  background-color: gray;
  color: white;
}
```

O código CSS deste exemplo define estilos para botões com três classes. A classe **.botao** estiliza o botão com padding, bordas arredondadas e remove a borda padrão, além de exibir um ponteiro ao passar o mouse. A classe **.ativo** aplica um fundo verde e texto branco para um botão ativo, enquanto a classe **.inativo** aplica um fundo cinza e texto branco para um botão inativo, diferenciando visualmente os estados de um botão.

Agora, no componente React, em **App.jsx**, vamos utilizar estas classes CSS:

```
import { useState } from "react";
import "./App.css"; // Importa as classes CSS
```

```
function App() {  
  const [ativo, setAtivo] = useState(false);  
  return (  
    <div>  
      <button  
        className={`botao ${ativo ? "ativo" : "inativo"}}`  
        onClick={() => setAtivo(!ativo)}>  
        {ativo ? "Ativo" : "Inativo"}  
      </button>  
    </div>  
  );  
}  
export default App;
```

Observe neste exemplo:

- **Combinação de Classes:** A classe base **botao** é sempre aplicada, mas as classes **ativo** ou **inativo** são alternadas com base no estado **ativo**.
- **Template Literals:** Usamos template literals para concatenar as classes dinamicamente.
- **Alternância de Classes:** Se **ativo** for **true**, a classe **ativo** será aplicada; caso contrário, a classe **inativo** será usada, alterando a cor do botão dinamicamente.

Veja na imagem abaixo a execução desta aplicação:

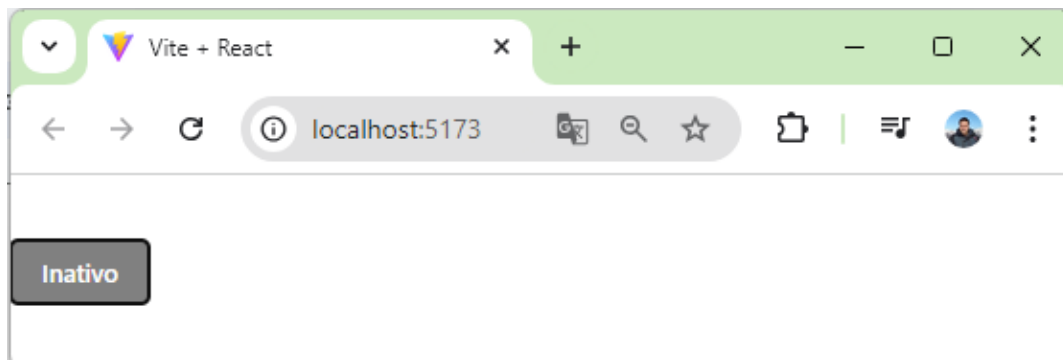


Figura 9: Exemplo sobre a aplicação de Classes CSS de forma condicional

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular cinza com o texto "Inativo" em branco. O restante da página tem fundo branco.

Em seguida, clique no botão "Inativo" e veja o resultado, conforme imagem abaixo:

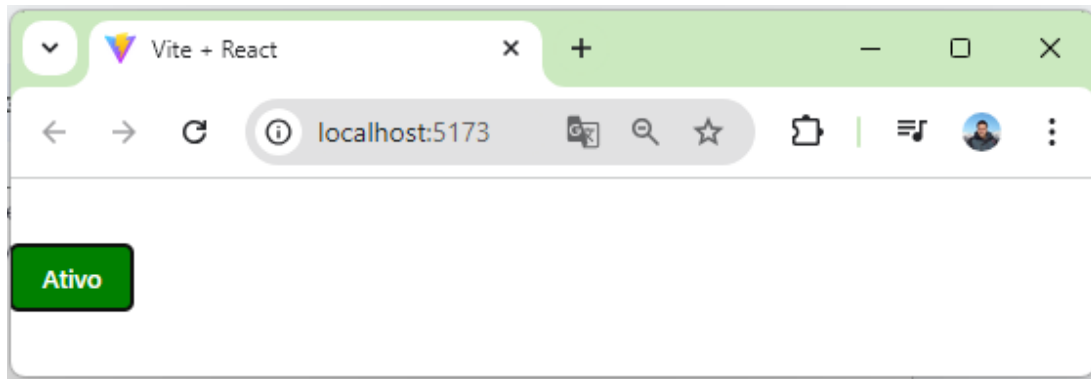


Figura 10: Exemplo sobre a aplicação de Classes CSS de forma condicional ao alterar o estado

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular verde com o texto "Ativo" em branco. O restante da página tem fundo branco.

3.3 Estilização Condicional com CSS Modules

CSS Modules escopam os estilos localmente, e você pode aplicar classes condicionalmente de forma semelhante ao uso de classes CSS tradicionais, mas com a vantagem de evitar conflitos de nomes. Vamos analisar a estilização condicional com CSS Modules através de um exemplo. Para isso, considere a criação de um arquivo **App.module.css** com o seguinte código:

```
.botao {  
  padding: 10px 20px;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}  
.ativo {  
  background-color: green;  
  color: white;  
}  
.inativo {  
  background-color: gray;  
  color: white;  
}
```

Em seguida, no componente React definido em **App.jsx**, considere o seguinte código:

```
import { useState } from "react";
import styles from "./App.module.css"; // Importa as classes CSS do CSS Module
function App() {
  const [ativo, setAtivo] = useState(false);
  return (
    <div>
      <button
        className={` ${styles.botao} ${ativo ? styles.ativo : styles.inativo}`}
        onClick={() => setAtivo(!ativo)}>
        {ativo ? "Ativo" : "Inativo"}
      </button>
    </div>
  );
}
export default App;
```

Observe neste código:

- **CSS Modules:** As classes CSS são importadas como um objeto JavaScript (**styles**), onde cada classe é uma chave do objeto.
- **Combinação de Classes:** Estamos combinando as classes com base no estado **ativo**, mas essas classes são escopadas localmente ao componente. Veja que dependendo do valor de **ativo**, o botão recebe a classe CSS correspondente, usando classes do CSS Module importadas de App.module.css.

Veja na imagem abaixo a execução desta aplicação:

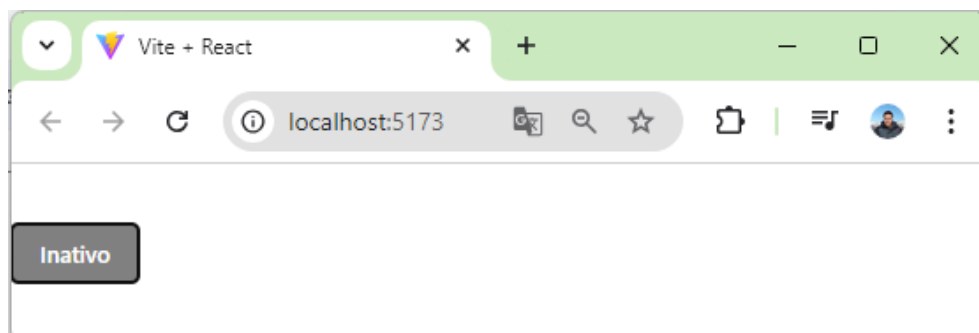


Figura 11: Exemplo sobre a aplicação de CSS Modules de forma condicional

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular cinza com o texto "Inativo" em branco. O restante da página tem fundo branco.

Em seguida, clique no botão "Inativo" e veja o resultado, conforme imagem abaixo:

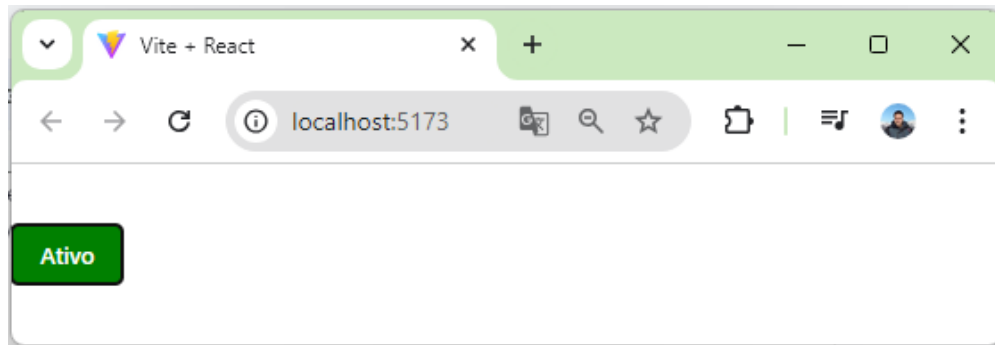


Figura 12: Exemplo sobre a aplicação de CSS Modules de forma condicional ao alterar o estado

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular verde com o texto "Ativo" em branco. O restante da página tem fundo branco.

3.4 Estilização Condicional com Styled Components

Com Styled Components, a estilização condicional se torna ainda mais poderosa, pois você pode aplicar estilos diretamente dentro do componente usando props e estado. Essa abordagem permite que você altere dinamicamente qualquer aspecto do estilo com base nas variáveis do componente. Veremos um exemplo prático de uso de estilização condicional com Styled Components. Para isso, considere o arquivo **App.jsx** com o seguinte código:

```
import { useState } from "react";
import styled from "styled-components";

// Definindo um botão estilizado com base em props
const Botao = styled.button`
  background-color: ${(props) => (props.ativo ? "green" : "gray")};
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  &:hover {
    background-color: ${(props) => (props.ativo ? "darkgreen" : "darkgray")};
  }
`;
```

```
function App() {  
  const [ativo, setAtivo] = useState(false);  
  
  return (  
    <div>  
      <Botao ativo={ativo} onClick={() => setAtivo(!ativo)}>  
        {ativo ? "Ativo" : "Inativo"}  
      </Botao>  
    </div>  
  );  
}  
export default App;
```

Observe neste exemplo:

- **Styled Components:** A biblioteca permite que você escreva estilos CSS diretamente dentro do componente JavaScript, usando **template literals**.
- **Estilos Baseados em Props:** A cor de fundo do botão muda dinamicamente com base no valor da **prop ativo**, que é derivada do estado. Se **ativo** for **true**, o botão será verde; caso contrário, será cinza.
- **Estilos Dinâmicos no hover:** A cor ao passar o mouse (**hover**) também muda dinamicamente com base no estado. O **&** é usado para referenciar o próprio elemento e adicionar pseudo-classes como **:hover**.

Veja na imagem abaixo a execução desta aplicação:

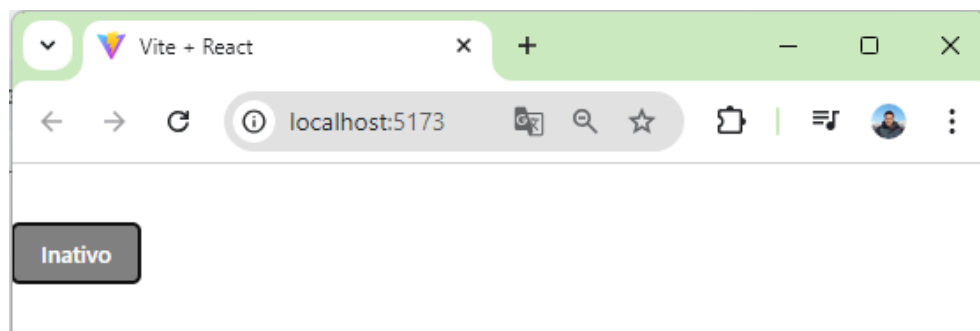


Figura 13: Exemplo sobre a aplicação de CSS Modules de forma condicional

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular cinza com o texto "Inativo" em branco. O restante da página tem fundo branco.

Em seguida, clique no botão "Inativo" e veja o resultado, conforme imagem abaixo:

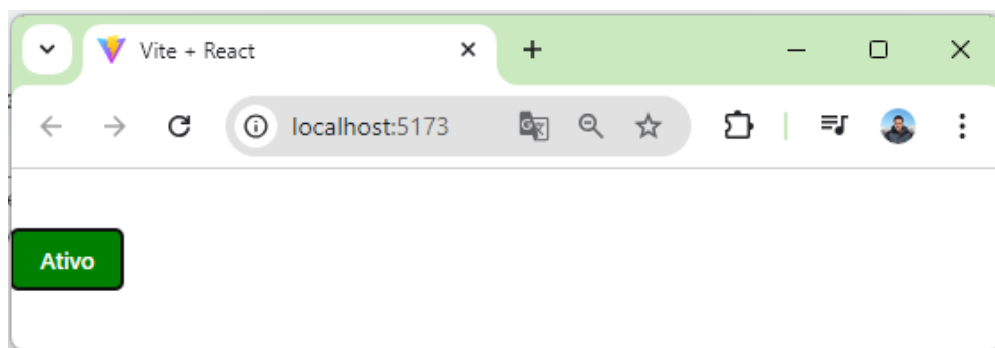


Figura 14: Exemplo sobre a aplicação de CSS Modules de forma condicional ao alterar o estado

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular verde com o texto "Ativo" em branco. O restante da página tem fundo branco.

4. Criação de Temas e Personalização de Interfaces em React

A criação de temas centraliza a lógica de estilo em um único lugar, permitindo alternar facilmente entre diferentes conjuntos de cores, fontes e espaçamentos. Existem várias abordagens para a criação de temas em React, mas a mais comum é usar a biblioteca **Styled Components** ou o **Context API**.

4.1 Utilizando Styled Components para Criar Temas

Styled Components é uma biblioteca popular para estilizar componentes em React, que permite criar componentes CSS em JavaScript. Uma de suas funcionalidades poderosas é a criação de temas, que facilita a troca de estilos em toda a aplicação de forma dinâmica e eficiente, como alternar entre um tema claro e tema escuro.

4.1.1 Como funciona a criação de temas com Styled Components?

A criação de temas com Styled Components funciona da seguinte forma:

- **Definição de tema:** Você cria um objeto com as cores, tamanhos, e outros atributos de estilo que deseja reutilizar.
- **ThemeProvider:** Esse componente especial do Styled Components permite envolver a aplicação e fornecer o tema para todos os componentes internos.
- **Acesso ao tema:** Dentro dos componentes estilizados, você pode acessar as propriedades do tema usando a função de interpolação **props.theme**.

4.1.2 Exemplo simples

Veremos um exemplo simples sobre a criação de temas com Styled Components. Dessa forma, primeiramente, considere a instalação da biblioteca Styled Components em seu projeto React através da execução do seguinte comando:

```
npm install styled-components
```

Em seguida, precisamos definir os temas. Para isso, considere a criação de um arquivo, chamado **theme.jsx**, para armazenar os diferentes temas. Por exemplo, considere um tema claro e um tema escuro.

```
export const lightTheme = {
  background: "#fff",
  color: "#000",
};
export const darkTheme = {
  background: "#000",
  color: "#fff",
};
```

Agora, use o **ThemeProvider** para passar o tema à aplicação. O **ThemeProvider** permite que qualquer componente estilizado possa acessar o tema via **props**. Assim, considere no arquivo **App.jsx** a seguinte lógica para alternar entre temas:

```
import { useState } from "react";
import { ThemeProvider } from "styled-components";
import { lightTheme, darkTheme } from "./theme";
import styled from "styled-components";

const Container = styled.div`
  background-color: ${(props) => props.theme.background};
  color: ${(props) => props.theme.color};
  height: 100vh;
  width: 100vw;
  display: flex;
  align-items: center;
  justify-content: center;
`;

function App() {
  const [isDarkMode, setIsDarkMode] = useState(false);
  return (
    <ThemeProvider theme={isDarkMode ? darkTheme : lightTheme}>
      <Container>
        <button onClick={() => setIsDarkMode(!isDarkMode)}>Trocar Tema</button>
      </Container>
    </ThemeProvider>
  );
}

export default App;
```

Neste exemplo, observe que:

- Dentro dos componentes estilizados, o tema é acessado através de **props.theme**. Aqui, **props.theme.background** e **props.theme.color** são usados para definir o fundo e a cor do texto dinamicamente com base no tema selecionado.
- **ThemeProvider**: Esse componente envolve toda a aplicação e fornece o tema selecionado para os componentes dentro dele. Isso cria um contexto de tema, de modo que qualquer componente pode acessar as propriedades do tema através de **props.theme**.
- **Troca de tema**: A troca entre o tema claro e o tema escuro é controlada pelo estado **isDarkMode**, que ao ser alterado muda o valor do tema passado ao **ThemeProvider**.

Veja na imagem abaixo a execução desta aplicação:

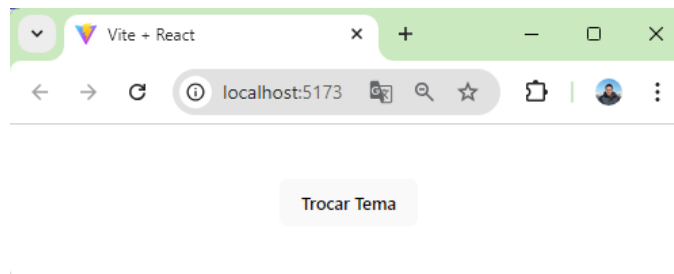


Figura 15: Exemplo sobre a criação de temas com Styled Components

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular cinza com o texto "Trocar Tema". O restante da página tem fundo branco.

Em seguida, clique no botão "Trocar Tema" e veja o resultado, conforme imagem abaixo:

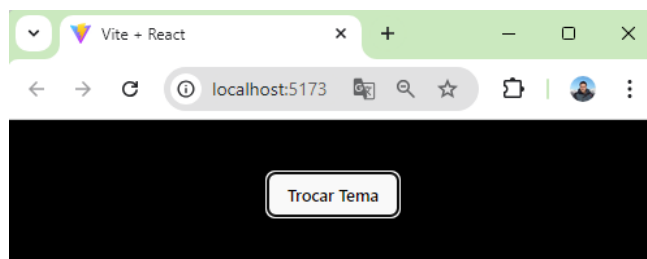


Figura 16: Exemplo sobre a aplicação de temas com Styled Components ao alterar o tema

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há um botão retangular cinza com o texto "Trocar Tema". O restante da página tem fundo preto.

4.2 Alternando Temas com Context API

Para alternar temas utilizando a Context API do React, podemos criar um contexto para armazenar e gerenciar o estado do tema globalmente. Isso permite que qualquer componente na aplicação acesse e altere o tema sem precisar passar props por vários níveis.

Primeiro, considere a criação de um arquivo chamado **theme.jsx** com os temas de nossa aplicação:

```
export const temaClaro = {
  background: "#fff",
  color: "#000",
};
export const temaEscuro = {
  background: "#000",
  color: "#fff",
};
```

Em seguida, considere a criação de um arquivo chamado **ThemeContext.jsx** com o seguinte código:

```
import { createContext, useState } from "react";
import { temaClaro, temaEscuro } from "../theme";

export const ThemeContext = createContext();

export const ThemeProvider = ({ children }) => {
  const [temaAtual, setTemaAtual] = useState(temaClaro);
  const alternarTema = () => {
    setTemaAtual(temaAtual === temaClaro ? temaEscuro : temaClaro);
  };
  return (
    <ThemeContext.Provider value={{ temaAtual, alternarTema }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

Este código cria um contexto de tema em React. Ele utiliza o **createContext** para criar o **ThemeContext** e um componente **ThemeProvider** que gerencia o estado do tema atual (**temaClaro** ou **temaEscuro**) usando **useState**. A função **alternarTema** alterna entre o tema claro e o escuro. O **ThemeProvider** envolve os componentes filhos e fornece o tema atual e a função para alternar o tema através do contexto, tornando-os acessíveis a qualquer componente que consuma o **ThemeContext**.

Em seguida, precisamos usar o contexto no componente React definido em **App.jsx** da seguinte forma:

```
import { useContext } from "react";
import { ThemeContext, ThemeProvider } from "../ThemeContext";
import "../App.css"; // Estilos tradicionais ou CSS Modules

function App() {
  const { temaAtual, alternarTema } = useContext(ThemeContext);
  return (
    <div
      style={{
        backgroundColor: temaAtual.background,
        color: temaAtual.color,
        minHeight: "100vh",
      }} >
      <h1>Aplicação com Temas e Context API</h1>
      <button
        style={{
          backgroundColor: temaAtual.background,
          color: temaAtual.color,
        }}
        onClick={alternarTema}>
        Alternar Tema
      </button>
    </div>
  );
}

export default () => (
  <ThemeProvider>
    <App />
  </ThemeProvider>
);
```

Observe que:

- **Contexto do Tema:** O **ThemeContext** mantém o estado do tema atual (**temaAtual**) e a função para alternar entre os temas.
- **useContext:** No componente **App**, usamos o hook **useContext** para acessar o tema atual e a função **alternarTema** diretamente.
- **Estilos Inline:** Aplicamos os estilos com base no tema diretamente no estilo inline dos elementos.

Veja na imagem abaixo a execução desta aplicação:

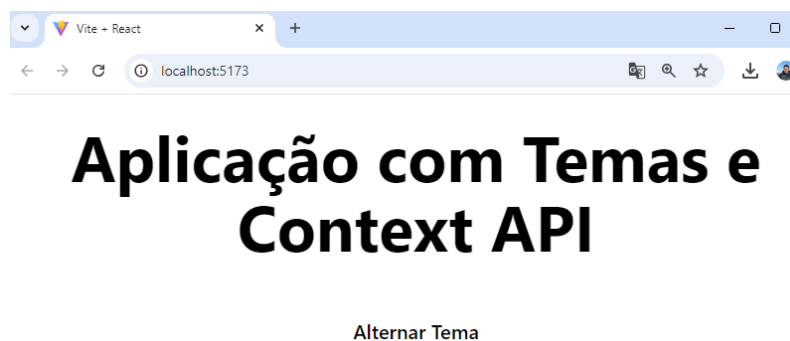


Figura 17: Exemplo sobre a criação de temas com Context API

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há o título em negrito "Aplicação com Temas e Context API" em preto. Abaixo, há um link ou botão com o texto "Alternar Tema". O fundo da página é branco.

Em seguida, clique no botão "Trocar Tema" e veja o resultado, conforme imagem abaixo:



Figura 18: Exemplo sobre a aplicação de temas com Context API ao alterar o tema

Descrição: A imagem mostra uma página web no navegador, com o título "Vite + React", rodando em "localhost:5173". No centro da página, há o título em negrito "Aplicação com Temas e Context API" em branco, com fundo preto. Abaixo, há um botão com bordas brancas e o texto "Alternar Tema".

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- DEVLIN BASILAN DULDULAO, D. B.; CABAGNOT, R. J. L. Practical Enterprise React: Become an Effective React Developer in Your Team. USA: APRESS, 2021.
- ELROM, E. React and Libraries: Your Complete Guide to the React Ecosystem. USA: APRESS, 2021.
- MDN WEB DOCS. Getting started with React. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started. Acessado em 21 jun. 2024.
- REACT DOCS. Disponível em <https://pt-br.react.dev/>. Acessado em 15 jun. 2024.
- ROLDÁN, C. S. React 17: Design Patterns and Best Practices. UK: Packt Publishing, 2021.
- SCOTT, A. D. JavaScript Everywhere. USA: O'Reilly Media, 2020.
- UZAYR, S. B.; CLOUD, N.; AMBLER, T. JavaScript Frameworks for Modern Web Development: The Essential Frameworks, Libraries, and Tools to Learn Right Now. USA: APRESS, 2019.
- W3SCHOOLS. React Tutorial. Disponível em: <https://www.w3schools.com/react>. Acessado em 17 jun. 2024.